

Begründung für die Implementierung: KatS-Stab-Dashboard

1. Zielgruppenanalyse

Im Rahmen des Katastrophenschutzes Berlin (Feuerwehr, THW, Rettungsdienste) ist die schnelle und gebündelte Erfassung von einsatzkritischen Lageinformationen unabdingbar. Ziel dieses Projektes war die Entwicklung eines internen Fach-Dashboards, das speziell auf diese operative Zielgruppe zugeschnitten ist. Da Einsatzkräfte oftmals unter Zeitdruck und starker kognitiver Belastung arbeiten, aggregiert das Dashboard verteilte Systemdaten auf einer einzigen, übersichtlichen Oberfläche. Der Fokus liegt dabei strikt auf Performance, Ausfallsicherheit und einer klaren Informationsarchitektur, um kritische Entscheidungen ohne Verzögerung zu ermöglichen.

2. Technische Randbedingungen

Eine zentrale technische Randbedingung war der effiziente Umgang mit externen Daten-Schnittstellen und den strikten Rate-Limits kostenfreier APIs (z. B. der Wetter-API, die bei dauerhaftem Abfragen blockiert). Um das System belastbar (resilient) zu machen, wurde eine **Smart Polling-Architektur** implementiert:

- **Idle-Sleep & Wakeup:** Das serverseitige Backend überwacht permanent die Anzahl der aktiven WebSocket-Clients. Sind null Nutzer verbunden, pausiert das Polling ("Schlafmodus"), um API-Kontingente zu schonen. Verbindet sich ein Client, wird ein asynchroner Trigger gefeuert und die Daten werden sofort aktualisiert.
- **Interne Cooldowns:** Strikte, service-spezifische Abklingzeiten (z.B. 15 Minuten Sperrzeit für Wetterdaten) garantieren, dass die zulässigen Request-Limits der Drittanbieter niemals überschritten werden. Dies ist notwendig, da die API selber keine Möglichkeit bereit stellt um festzustellen ob die Daten aktualisiert wurden, sondern nur komplette Datensätze schicken.

3. Datenquellen

Gemäß der Anforderung, realistische und valide Live-Daten abzubilden, wurden ausschließlich offizielle Schnittstellen angebunden:

- **Wetter (Open-Meteo / DWD-ICON):** Die Wahl fiel gezielt auf die Daten des Deutschen Wetterdienstes, um höchste lokale Präzision (7-Tage-Prognose, UV-Index, offizielle Warnstufen) für Berlin sicherzustellen.

- **Pegelstände (PEGELONLINE - WSV):** Die REST-API der Wasserstraßen- und Schifffahrtsverwaltung wurde gewählt, um Berliner Messstellen (inklusive *BERLIN-KÖPENICK*) dynamisch abzufragen und historische Trends in einem 24h-Chart abzubilden.
- **Einsatzdaten (Berliner Feuerwehr):** Statt auf statische Mock-Daten zurückzugreifen, wird ein täglicher CSV-Dump aus dem Open-Data-GitHub der Feuerwehr automatisiert ausgelesen und die Einsätze des Vortags kategorisiert.

4. Technologien

Um Skalierbarkeit und Austauschbarkeit zu gewährleisten, wurde eine **strikte Trennung von Frontend und Backend** beschlossen. Dies entlastet die Endgeräte der Einsatzkräfte, da das Polling komplett gesondert auf einem Server erfolgt.

- **Backend (Python, FastAPI, Uvicorn, WebSockets):** *FastAPI* und der Server *Uvicorn* bilden den modernen Industriestandard für asynchrone Anwendungen. Manche Frameworks (wie *Flask*) sind nicht im Hinblick auf Async gebaut worden. Clients und Server kommunizieren mit WebSocket-Verbindungen, um neue Datenpakete ohne Verzögerung via. Push-Verfahren an die Clients zu senden, statt diese zu permanenten HTTP-Polling zu zwingen.
- **Frontend (React):** Die Wahl fiel auf React aufgrund der komponentenbasierten Architektur und der enormen Marktdurchdringung. Letzteres erleichtert das Onboarding neuer Entwickler für künftige Dashboard-Erweiterungen immens.

5. Details zur Benutzerschnittstelle

Die Benutzeroberfläche wurde als Single-Page-Application konzipiert und mit **Tailwind CSS** sowie **Recharts** (für Diagramme wie die Einsatz-Treemap und Pegel-Historie) realisiert.

- **Design & Accessibility:** Es wurde bewusst ein **Light-Theme** gewählt, um die Kontraste gemäß den Accessibility-Anforderungen für operative Einsatzkräfte zu maximieren.
- **Interaktivität & Fehlerbehandlung (Resilienz):** Die UI ist interaktiv (z. B. aufklappbare Wetter-Details). Bei Ausfällen einer Schnittstelle fängt das Frontend die Fehler ab und rendert dezente rote Fehler-Banner, anstatt abzustürzen. Um der Zielgruppe jederzeit die volle Kontrolle zu geben, verfügt jedes Widget über einen manuellen Refresh-Button, mit dem Aktualisierungen in kritischen Lagen gezielt erzwungen werden können.